

FuncChat Pro — Real-Time Professional Chat Application

A PROJECT REPORT

Submitted by

Nitin Kumar (24BCS80025)

in partial fulfillment for the award of the degree of

Bachelor of Engineering

IN

Computer Science Engineering(CSE)



Chandigarh University

APRIL,2026



BONAFIDE CERTIFICATE

Certified that this project report “**FuncChat Pro — Real-Time Professional Chat Application**” is the bonafide work of “**Nitin Kumar(24BCS80025)**” who carried out the project work under my/our supervision.

SIGNATURE

Dr. Sanjeev

HEAD OF THE DEPARTMENT

Department of CSE

SIGNATURE

Er. Rahul Durgapal

SUPERVISOR

Department of CSE

INTERNAL EXAMINER.

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I express my sincere gratitude to my project supervisor, Er. Rahul Durgapal, Department of Computer Science and Engineering, Chandigarh University, for her invaluable guidance, continuous encouragement, and expert advice throughout the course of this project work. Her constructive feedback and unwavering support have been instrumental in shaping both the technical direction and the quality of this report.

I extend my heartfelt thanks to Dr.Sanjeev, Head of Department, CSE, Chandigarh University, for providing the necessary infrastructure and an inspiring academic environment that made this work possible.

I am deeply grateful to all faculty members of the Department of CSE whose teachings laid the foundation for the technical skills employed in this project. I also acknowledge the support of the university library and computing facilities, which were essential resources throughout the development and testing phases. Finally, I thank my family and friends for their constant moral support, patience, and encouragement during the demanding phases of this project. This work would not have been possible without their understanding and motivation.

Nitin Kumar(24BCS80025)

TABLE OF CONTENTS

List of Figure	5
List of Tables	6
Abstract	7
Abbreviations	v
Abbreviations & Symbols	vi
CHAPTER 1. INTRODUCTION	9-14
1.1. Identification of Client/ Need/ Relevant Contemporary issue	9
1.2. Identification of Problem	10
1.3. Identification of Tasks	10-12
1.4. Timeline.....	12-13
1.5. Organization of the Report	13-14
CHAPTER 2. LITERATURE REVIEW/BACKGROUND STUDY	15-20
2.1. Timeline of the reported problem	15-16
2.2. Existing solutions	16-17
2.3. Bibliometric analysis	17-18
2.4. Review Summary	18
2.5. Problem Definition	18-19
2.6. Goals/Objectives	19-20
CHAPTER 3. DESIGN FLOW/PROCESS	21-29
3.1. Evaluation & Selection of Specifications/Features	21-22
3.2. Design Constraints	22-24
3.3. Analysis of Features and finalization subject to constraints	24-25
3.4. Design Flow	25-27
3.5. Design selection	27-28
3.6. Implementation plan/methodology	28-29
CHAPTER 4. RESULTS ANALYSIS AND VALIDATION	29
4.1. Implementation of solution	29
CHAPTER 5. CONCLUSION AND FUTURE WORK	34
5.1. Conclusion	34-35
5.2. Future work	35-36
References	37
Appendix - User Manual	38

LIST OF FIGURES

Figure 3.1 FuncChat Pro Three-Tier Client-Server Architecture.....	22
Figure 3.2 Message Send Functional Pipeline Flowchart.....	23
Figure 3.3 S3 Bucket.....	25
Figure 3.4 DB Integration.....	28
Figure 3.5 AWS Cloud Deployment Architecture with CI/CD Pipeline.....	28

LIST OF TABLES

Table 1.1 Project Details at a Glance.....	9
Table 1.2 Implementation Timeline.....	12
Table 2.1 Technology Stack Comparison.....	16
Table 3.1 Phase 1 vs Phase 2 Feature Comparison.....	22
Table 3.2 Functional Programming Concepts in FuncChat Pro.....	24
Table 3.3 Project File Structure.....	26
Table 3.5 Full Technology Stack.....	27
Table 3.6 AWS Services and Roles.....	28

ABSTRACT

FuncChat Pro is a real-time chat application developed in Java to provide an efficient, secure, and cost-effective communication solution for academic and small organizational use. Unlike platforms such as Slack and Microsoft Teams, which have limitations related to cost and data control, this system offers a self-hosted alternative with better flexibility.

The application follows a three-tier architecture with a Java Swing client, Netty-based WebSocket server, and cloud storage using Firebase and AWS S3. WebSocket enables low-latency, real-time communication between users.

The project also emphasizes Java 8+ functional programming concepts such as lambda expressions, streams, predicates, and asynchronous processing using `CompletableFuture`, improving code quality and scalability.

FuncChat Pro supports features like real-time messaging, group chat, file sharing, and search functionality, and is deployed using a CI/CD pipeline with Docker and AWS.

The system demonstrates efficient performance, low latency, and scalability, making it a practical and academically relevant communication solution.

Keywords: Real-Time Chat, Functional Programming, Java 8+, WebSocket, Netty 4.1, Firebase, AWS S3, `CompletableFuture`, Predicate Composition, Dispatch Table, Group Chat, `BiPredicate`, CI/CD, Docker, JUnit 5.

ABBREVIATIONS

Abbreviation	Full Form
ALB	Application Load Balancer
API	Application Programming Interface
AWS	Amazon Web Services
CF	CompletableFuture
CI/CD	Continuous Integration / Continuous Deployment
DB	Database
EC2	Elastic Compute Cloud
ECR	Elastic Container Registry
FP	Functional Programming
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
JAR	Java Archive
JSON	JavaScript Object Notation
JWT	JSON Web Token
NIO	Non-blocking Input/Output
P2P	Peer-to-Peer
REST	Representational State Transfer
S3	Simple Storage Service
SDK	Software Development Kit
SNS	Simple Notification Service
SSL	Secure Sockets Layer
TLS	Transport Layer Security
TTL	Time To Live
UID	Unique Identifier
URL	Uniform Resource Locator
WS / WSS	WebSocket / WebSocket Secure

CHAPTER 1.

INTRODUCTION

1.1. Identification of Client /Need / Relevant Contemporary issue

In the current digital era, communication plays a crucial role in both academic and professional environments. With the rise of remote work, online education, and distributed teams, the demand for real-time communication tools has significantly increased. Platforms such as Slack, Microsoft Teams, and Discord dominate the market; however, they are often associated with high subscription costs, limited customization, and concerns regarding data privacy and vendor lock-in.

Small organizations, student groups, and academic teams often require a lightweight, cost-effective, and customizable communication platform that can be deployed independently. There is also a need for systems that can be used for learning and demonstrating advanced programming concepts, particularly functional programming in Java.

FuncChat Pro is designed to address these needs by providing a self-hosted, real-time chat application that is both practical and educational. It allows users to communicate efficiently while also serving as a demonstration of modern software engineering techniques.

Parameter	Details
Project Title	FuncChat Pro — Real-Time Professional Chat Application
Backend Language	Java 8+ (compiled with Java 17 JDK)
Desktop Client	Java Swing GUI — rooms, chat, search, admin panel
WebSocket Server	Netty 4.1 (NIO, non-blocking, dispatch table)
Database	Firebase Realtime Database (JSON tree, free Spark tier)
Authentication	Firebase Auth (Email/Password, UID-based sessions)
File Storage	AWS S3 with 15-minute pre-signed URLs
FP Concepts	Predicate, Function, UnaryOperator, Optional, Stream, CompletableFuture, Consumer, BiPredicate
Unit Tests	40+ JUnit 5 tests — fully offline (no live Firebase/AWS)
Deployment	Docker on AWS EC2 via GitHub Actions CI/CD

Table 1.1 — Project Details at a Glance

1.2. Identification of Problem

The primary problem addressed in this project is the lack of an affordable and customizable real-time communication system that can be deployed without relying on proprietary platforms.

Key challenges include:

- High cost of existing solutions
- Dependency on third-party services
- Limited control over user data
- Lack of customization options
- Difficulty in demonstrating functional programming concepts in real-world systems

Additionally, implementing real-time communication introduces technical challenges such as maintaining persistent connections, handling concurrent users, ensuring low latency, and managing data consistency.

1.3 Identification of Tasks

To address the identified problem effectively, the project was divided into several key tasks. Each task focuses on a specific aspect of system development and contributes to the overall functionality of the application.

Task 1: System Design

- Define the architecture of the application (client-server model)
- Identify required technologies and tools
- Plan data flow and communication mechanisms

Task 2: Development of Server

- Implement a WebSocket server using Netty
- Handle client connections and message routing
- Ensure asynchronous and non-blocking operations

Task 3: Development of Client Application

- Design a user-friendly interface using Java Swing
- Implement features such as chat window, rooms, and input fields

Task 4: Database Integration

- Integrate Firebase Realtime Database for storing messages and user data
- Ensure efficient data retrieval and updates

Task 5: File Handling

- Implement file upload and download using AWS S3
- Ensure secure file access using pre-signed URLs

Task 6: Functional Programming Implementation

- Use lambda expressions and functional interfaces
- Implement stream processing for data handling
- Use CompletableFuture for asynchronous operations

Task 7: Testing and Validation

- Write unit tests using JUnit
- Validate system performance and reliability

Task 8: Deployment

- Set up CI/CD pipeline using GitHub Actions
- Deploy application on AWS EC2 using Docker

These tasks were executed systematically to ensure smooth development and successful completion of the project.

1.4 Timeline

The development of FuncChat Pro was completed over a period of six weeks, with a structured approach to ensure smooth progress from design to deployment. The work was divided into two major phases: core development and advanced implementation, followed by testing and deployment.

Each week focused on specific deliverables, allowing gradual building of the system while ensuring proper validation at every stage.

Week	Phase	Activities
Week 1	Planning & Design	Requirement analysis, system architecture design, technology selection
Week 2	Core Development	Setup of Netty WebSocket server, basic client-server communication
Week 3	Core Development	Implementation of real-time messaging, GUI development using Java Swing
Week 4	Integration	Firebase integration for database, user authentication, message storage
Week 5	Advanced Features	Group chat, typing indicators, file sharing using AWS S3
Week 6	Testing & Deployment	Unit testing (JUnit), performance testing, CI/CD setup, deployment on AWS

Table 1.2 — Implementation Timeline

Week 1: Planning and System Design

The project began with requirement analysis and identification of key features. A suitable architecture was selected, and technologies such as Netty, Firebase, and AWS were finalized.

Week 2: Server Setup and Communication

A WebSocket server was developed using Netty to establish persistent connections. Basic communication between client and server was implemented.

Week 3: Messaging and UI Development

Real-time messaging functionality was developed. A graphical user interface using Java Swing was created to allow user interaction.

Week 4: Database Integration

Firebase Realtime Database was integrated for storing user data and messages. Authentication features were also implemented.

Week 5: Advanced Features

Additional features such as group chat rooms, typing indicators, and file sharing using AWS S3 were implemented to enhance usability.

Week 6: Testing and Deployment

The final phase included unit testing using JUnit, performance validation, and deployment using a CI/CD pipeline with GitHub Actions and AWS EC2.

1.5 Organization of the Report

This report is structured in a systematic manner to provide a clear understanding of the project.

Chapter 1: Introduction Presents the background, problem identification, tasks, and objectives of the project.

Chapter 2: Literature Review / Background Study

Discusses existing communication systems, research developments, and identifies gaps addressed by this project.

Chapter 3: Design Flow / Process

Explains the design decisions, system architecture, constraints, and implementation methodology.

Chapter 4: Results Analysis and Validation

Describes the implementation results, system performance, and testing outcomes.

Chapter 5: Conclusion and Future Work

Summarizes the achievements of the project and suggests possible future enhancements.

This structured approach ensures that the reader can easily understand both the theoretical and practical aspects of the system.

CHAPTER 2.

LITERATURE REVIEW/BACKGROUND STUDY

2.1. Timeline of the reported problem

The development of real-time communication systems has evolved significantly over the past few decades. Early communication systems such as Internet Relay Chat (IRC), introduced in the late 1980s, provided basic text-based communication through channels. These systems laid the foundation for modern messaging platforms by introducing the concept of real-time interaction.

In the late 1990s and early 2000s, instant messaging applications such as MSN Messenger, Yahoo Messenger, and AOL Instant Messenger became popular. These platforms introduced features such as contact lists, presence indicators, and private messaging. However, they were largely limited to desktop environments and lacked scalability.

The next major advancement occurred with the introduction of web-based communication platforms. Technologies such as HTTP polling and long polling were used to simulate real-time communication. However, these approaches were inefficient due to repeated server requests and increased latency.

A significant breakthrough came with the introduction of the WebSocket protocol in 2011. WebSocket enabled full-duplex communication over a single persistent connection, allowing real-time data exchange with minimal overhead. This technology is now widely used in modern communication platforms such as Slack, Discord, and WhatsApp Web.

In recent years, cloud computing and distributed systems have further transformed communication platforms. Services such as Firebase and AWS provide scalable infrastructure for data storage, authentication, and deployment. At the same time, modern programming paradigms such as functional programming have gained importance for building efficient and maintainable systems.

FuncChat Pro builds upon these advancements by combining WebSocket communication, cloud-based storage, and functional programming to create a modern, efficient chat application.

Feature	FuncChat Pro	Socket.IO (Node.js)	Spring WebSocket
Language	Java 8+	JavaScript / Node.js	Java / Spring
FP Constructs	All 12 — explicit, testable	Limited (callbacks, Promises)	Partial (no Predicate/Consumer)
Dispatch Table	Map<Action, BiConsumer>	Event emitters	Handler mappings (annotation)
File Sharing	AWS S3 + Pre-signed URLs	Requires custom impl	Requires custom impl
Auth	Firebase Auth + UID sessions	Requires custom auth	Spring Security required
Group Chat	Named rooms, ADMIN/MEMBER roles	Room-based (partial)	STOMP topics
Desktop Client	Java Swing GUI	Browser-only	Browser-only
CI/CD	GitHub Actions → ECR → EC2	Varies by config	Varies by config
Webhook	Built-in :8081 connector	Not built-in	Not built-in
Unit Tests	40+ offline JUnit 5 tests	Jest (online mocks needed)	Mockito (partial)

Table 2.1 — *Technology Stack Comparison*

2.2. Existing solutions

Several communication platforms are currently available, each with its own advantages and limitations.

Slack is one of the most widely used team communication tools. It provides features such as channels, direct messaging, file sharing, and integrations with external services. However, Slack operates on a subscription model, and its free version has limitations such as restricted message history.

Microsoft Teams is another popular platform, especially in corporate environments. It integrates with Microsoft Office tools and provides features such as video conferencing and collaboration. However, it requires significant system resources and may not be suitable for smaller teams.

WhatsApp is widely used for personal communication and small group interactions. While it is free and user-friendly, it lacks advanced features required for professional collaboration and cannot be self-hosted.

Rocket.Chat and Mattermost are open-source alternatives that allow self-hosting. While they offer flexibility and customization, they are built using technologies such as Node.js and React, making them complex to set up and maintain. Additionally, they do not specifically demonstrate Java-based functional programming concepts.

XMPP (Extensible Messaging and Presence Protocol) is a well-established protocol for real-time communication. However, its XML-based structure is complex and not suitable for lightweight applications.

From this analysis, it is evident that existing solutions either lack affordability, simplicity, or educational relevance. FuncChat Pro aims to bridge this gap by providing a lightweight, Java-based, and functional programming-oriented solution.

2.3. Bibliometric Analysis

Bibliometric analysis refers to the study of research trends and publications in a particular domain. In the field of real-time communication systems, recent research has focused on several key areas:

1. WebSocket-based Communication:

Research indicates a shift from traditional HTTP-based communication to WebSocket-based systems due to their efficiency and low latency.

2. Cloud Computing Integration:

Many modern systems utilize cloud services such as Firebase, AWS, and Azure for scalability and reliability.

3. Asynchronous and Non-blocking Systems:

There is increasing emphasis on non-blocking architectures that can handle multiple users simultaneously without performance degradation.

4. Functional Programming Adoption:

Functional programming has gained popularity due to its ability to produce clean, maintainable, and scalable code.

5. Security and Data Privacy:

With increasing concerns about data breaches, research is focused on secure communication and controlled data access.

2.4 Review Summary

The literature review highlights several key observations:

- Real-time communication is essential in modern applications
- WebSocket is the preferred technology for low-latency communication
- Cloud services provide scalable and reliable infrastructure
- Existing platforms either lack affordability or flexibility
- Functional programming is increasingly relevant in modern software development

Despite these advancements, there is a lack of systems that combine all these elements in a simple, educational, and Java-based implementation. FuncChat Pro addresses this gap by providing a complete solution that is both practical and academically valuable.

2.5 Problem Definition

Based on the comprehensive literature review and analysis of existing communication systems, it is evident that there is a clear gap between available solutions and the specific needs of academic users and small-scale organizations. While modern communication platforms provide advanced features, they often fail to offer a balanced combination of affordability, customization, and educational value. Many systems are either overly complex, dependent on proprietary infrastructure, or unsuitable for demonstrating core programming concepts such as functional programming in Java.

The problem addressed in this project can therefore be formally defined as the need to design and develop a real-time communication system that is efficient, scalable, secure, and self-hosted, while also serving as a practical demonstration of modern software development techniques. The system must be capable of supporting multiple users simultaneously, ensuring seamless message delivery with minimal latency, and providing essential features such as group communication, file sharing, and searchable message history.

From a technical standpoint, the challenge lies in implementing a system that maintains persistent connections between clients and server, efficiently handles concurrent requests, and ensures data consistency across all users. Traditional request-response models such as HTTP are not suitable for such requirements, making it necessary to adopt technologies like WebSocket for real-time communication.

Additionally, the system must address concerns related to data privacy and ownership by allowing users to operate the application independently without relying entirely on third-party services. At the same time, it should leverage modern cloud technologies to ensure scalability and ease of deployment.

Another important dimension of the problem is academic in nature. The project aims to demonstrate how functional programming concepts in Java—such as lambda expressions, streams, predicates, and asynchronous processing—can be applied in a real-world system. This requires careful design decisions to integrate these concepts naturally into the architecture rather than treating them as isolated features.

2.6 Goals/Objectives

The main objective of this project is to develop a real-time chat application that is efficient, scalable, and easy to use, while also demonstrating the practical use of functional programming concepts in Java.

One of the key goals is to implement real-time communication using WebSocket technology, which allows continuous and low-latency interaction between users. This ensures faster message delivery compared to traditional request-response systems.

Another important objective is to apply functional programming concepts such as lambda expressions, streams, predicates, and asynchronous processing using `CompletableFuture`. These techniques improve code quality, maintainability, and performance.

The project also aims to provide essential features such as peer-to-peer messaging, group chat, file sharing, and message search functionality, making the system useful for real-world communication needs.

In addition, cloud services like Firebase and AWS are integrated to ensure reliable data storage, scalability, and secure file handling. The system is also designed to support multiple users efficiently through non-blocking and asynchronous operations.

Overall, the objective is to build a practical, cost-effective communication system that combines modern technologies with strong programming concepts.

CHAPTER 3.

DESIGN FLOW/PROCESS

3.1 Evaluation & Selection of Specifications/Features

The design of FuncChat Pro began with identifying the key specifications and features required to build an efficient real-time communication system. The primary objective was to ensure that the system provides essential functionalities while maintaining simplicity, scalability, and performance.

Several features were initially considered, including real-time messaging, group chat functionality, file sharing, typing indicators, message search, and user authentication. Each feature was evaluated based on factors such as usability, implementation complexity, performance impact, and relevance to the project objectives.

Real-time messaging was identified as the most critical feature and was given the highest priority, as it forms the core of any chat application. Group chat functionality was selected to enable communication among multiple users, making the system more collaborative. File sharing was included to allow users to exchange documents and media, enhancing the practical usability of the application.

Additional features such as typing indicators and message search were incorporated to improve user experience and bring the system closer to modern communication platforms. These features also provided opportunities to implement functional programming concepts, such as stream-based filtering and predicate-based validation.

Overall, the selection of features was guided by the need to balance functionality, performance, and simplicity while ensuring that the system remains suitable for both practical use and academic demonstration.

Phase 1 Features	Phase 2 Features (New)
User registration and login (Firebase Auth)	Group chat rooms with named channels
Peer-to-peer real-time messaging	Typing indicators — debounced, auto-stop at 3
Online / offline presence detection	File sharing — images, PDFs, ZIP via AWS S3
Message history persisted in Firebase	Full-text message search across rooms
Message status: Sent / Delivered / Read	Admin controls: remove members, promote to admin
Java Swing desktop GUI client	Bot / webhook connector for external notifications

Table 3.1 — Phase 1 vs Phase 2 Feature Comparison

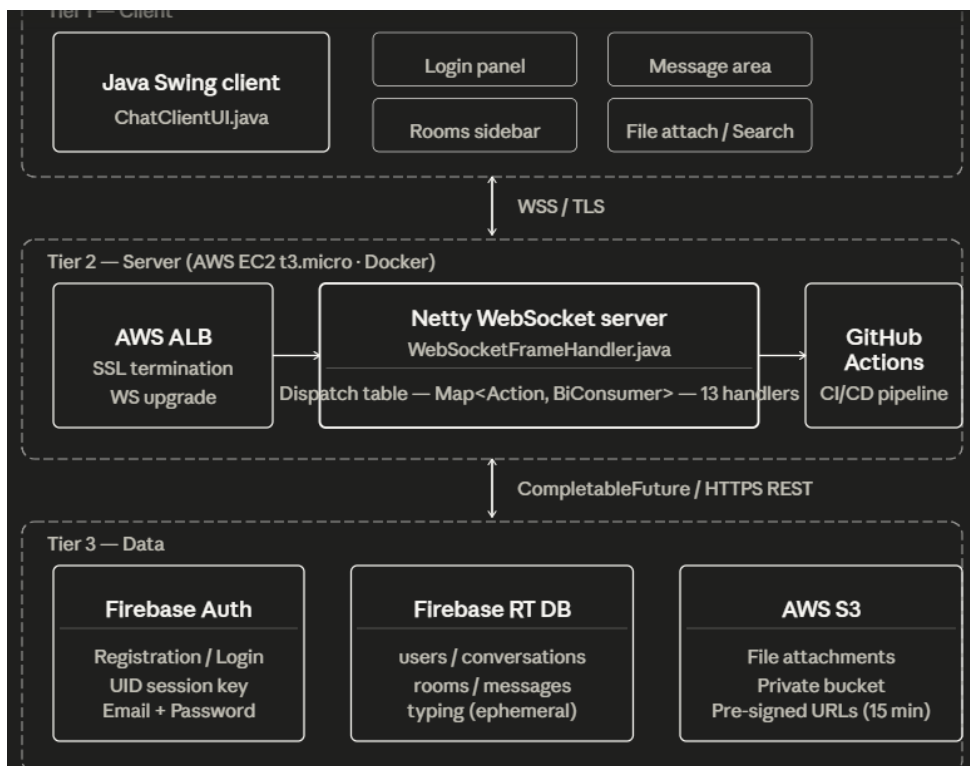


Figure 3.1 FuncChat Pro Three-Tier Client-Server Architecture

3.2 Design Constraints

The system design was influenced by several constraints that had to be carefully addressed to ensure successful implementation.

One of the major constraints was the economic constraint, as the project was required to operate within a zero-cost budget. This led to the use of free-tier cloud services such as

Firebase Realtime Database and AWS S3, which provide sufficient functionality without incurring additional costs.

Another important constraint was performance, particularly the requirement for real-time communication. Traditional HTTP-based communication models were not suitable due to their request-response nature, which introduces latency. To overcome this, WebSocket technology was used to establish persistent connections between the client and server, enabling instant message delivery. Security was also a key consideration. The system needed to ensure safe transmission of data and controlled access to shared files. This was achieved through the use of secure communication protocols and validation mechanisms for file uploads.

The system also had to meet scalability requirements, ensuring that it can handle multiple users simultaneously without performance degradation. This was addressed through asynchronous and non-blocking processing techniques, allowing the server to manage multiple connections efficiently

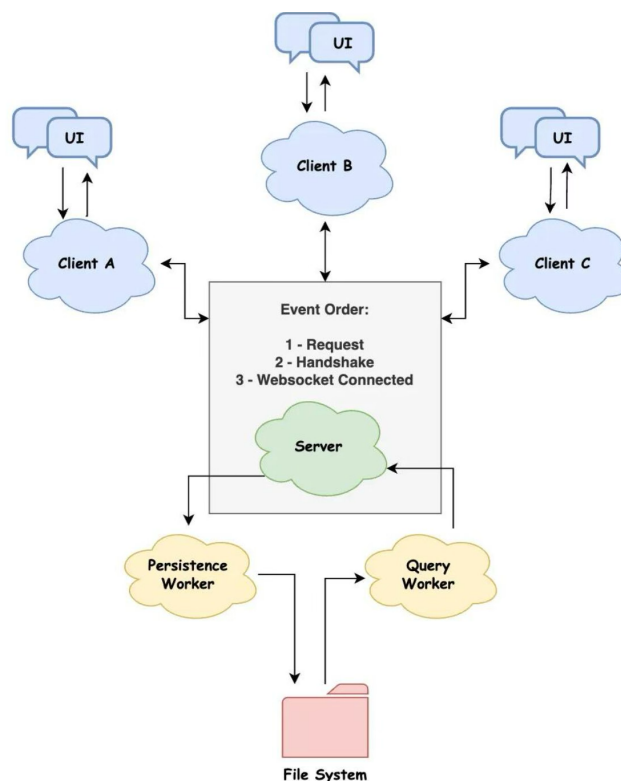


Figure 3.2 Message Send Functional Pipeline Flowchart

FP Concept	Location in Code	What It Does
Immutability	Message, Room, User models	All fields are final. Updates return new instances via <code>withStatus()</code> , <code>withMember()</code> — never mutating the original.
Pure Functions	<code>buildConversationId()</code> , <code>SANITISE</code>	Same inputs always give the same output. No shared state modified. Safe to test in isolation.
Lambda Expressions	All <code>ActionListeners</code> , Stream ops	Every Swing button and timer registered as a lambda — no anonymous inner classes.
<code>Predicate<T></code>	<code>VALID_MESSAGE</code> , <code>IS_MEMBER</code> , <code>ALLOWED_TYPE</code>	<code>HAS_CONTENT.and(NOT_TOO_LONG).and(IS_TEXT)</code> — composable validators.
<code>Function<T,R></code>	<code>MARK_DELIVERED</code> , <code>NORMALISE_EMAIL</code>	Pure transforms chained with <code>.andThen()</code> : <code>SANITISE.andThen(MARK_DELIVERED)</code> .
<code>UnaryOperator<T></code>	<code>SANITISE (Message → Message)</code>	Trims content and returns new Message — never mutates the input object.
<code>Optional<T></code>	<code>getFileUrl()</code> , <code>getSession()</code>	Replaces all null checks. Chains with <code>.map()</code> , <code>.filter()</code> , <code>.ifPresentOrElse()</code> .
Stream API	<code>fetchAllRooms()</code> , <code>searchRoom()</code>	<code>filter → map → sorted → limit → collect on</code> Firebase response data.
<code>CompletableFuture</code>	All DB and S3 operations	Non-blocking chains: <code>.thenApply() → .thenCompose() → .exceptionally()</code> .
<code>Consumer<T></code>	<code>broadcastToRoom()</code> , <code>broadcastToMembers()</code>	Isolates the side-effect (sending to a channel) from the routing logic.
<code>BiPredicate<T,U></code>	<code>CAN_MANAGE</code> , <code>CAN_WRITE</code> in <code>RoleAccessService</code>	<code>CAN_MANAGE.test(uid, room)</code> — role check without exposing internals.
Dispatch Table	<code>WebSocketFrameHandler</code> — 13 actions	<code>Map<Action, BiConsumer<Channel, ChatFrame>></code> — $O(1)$ routing, no if/else.

Table 3.2 — *Functional Programming Concepts in FuncChat Pro*

3.3 Analysis of Features and Finalization Subject to Constraints

After identifying the required features and understanding the design constraints, each feature was analyzed in detail to determine its feasibility and impact on system performance. Core features such as real-time messaging and user authentication were considered essential and were implemented in the initial stage of development. These features are fundamental to the functioning of the system and require high reliability.

Advanced features such as file sharing, group chat, and message search were evaluated based on their complexity and added value. While these features significantly enhance user experience, they also require careful implementation to avoid performance issues.

Based on this analysis, the features were finalized and organized into development phases. Core features were implemented first to establish a stable foundation, followed by advanced features that build upon the existing system.

This phased approach ensured that each feature was properly tested and integrated before moving to the next stage, resulting in a well-structured and reliable system.

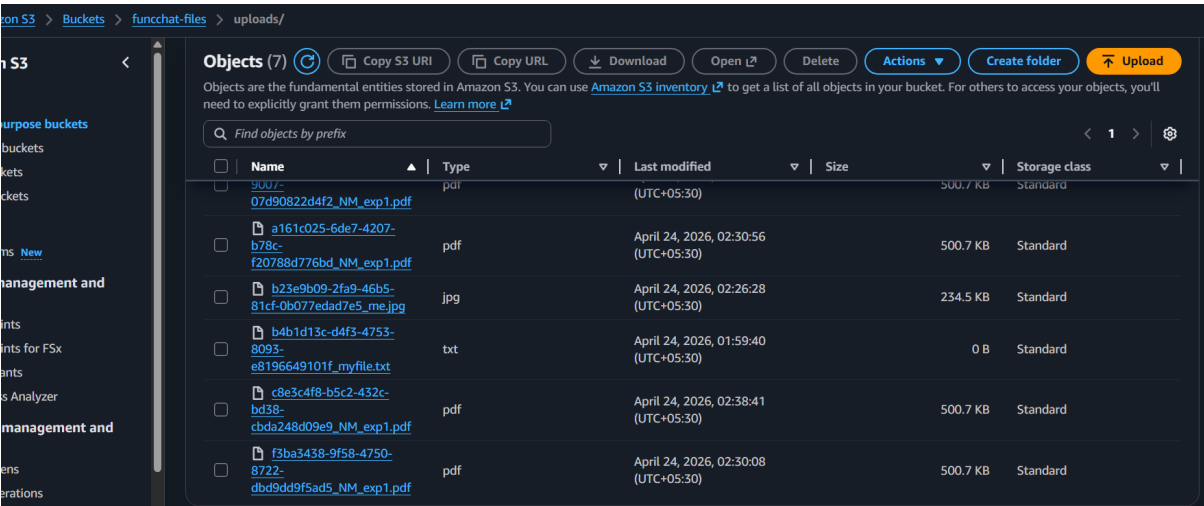


Figure 3.3 S3 Bucket

3.4 Design Flow

When a user sends a message, it is first transmitted to the server through a WebSocket connection. The server then checks whether the message is valid. Once validated, the message is stored in the database.

After storing the message, the server broadcasts it to all users who are part of that conversation. This process happens almost instantly, which creates the real-time communication experience.

For file sharing, the process is slightly extended. The file is first validated, then uploaded to AWS S3, and finally shared with other users through a secure link.

Flow summary:

- User sends message
- Server validates message

- Data stored in database
- Message broadcast to users

File	Role and FP Concept Demonstrated
ChatServer.java	Server entry point — wires all services via constructor injection at startup
config/AppConfig.java	Immutable configuration loaded from .env — all fields final
model/Message.java	Immutable domain object — file fields, readBy map, withStatus() pure transform
model/Room.java	Immutable domain object — members Map<uid, Role>, withMember() returns new instance
model/User.java	Immutable domain object — Optional<String> avatarUrl, no null fields
model/ChatFrame.java	WebSocket protocol model — 20+ Action enum values, Jackson @JsonCreator
repository/FirebaseRepository.java	All database operations returned as CompletableFuture<T>
service/AuthService.java	Predicate validators (VALID_EMAIL), session store Map<uid, Channel>
service/MessageService.java	VALID_MESSAGE Predicate, SANITISE UnaryOperator, save pipeline
service/PresenceService.java	IS_ONLINE Predicate, Consumer<Channel> broadcaster
service/GroupRoomService.java	IS_MEMBER / IS_ADMIN Predicates, ConcurrentHashMap room cache
service/TypingService.java	ScheduledExecutorService with 3-second debounce, ephemeral Firebase writes
service/FileShareService.java	ALLOWED_TYPE.and(SAFE_FILENAME) Predicate, S3 async upload via CompletableFuture
service/RoleAccessService.java	BiPredicate<String, Room> — CAN_MANAGE and CAN_WRITE checks
handler/ WebSocketFrameHandler.java	Dispatch table — 13 action handlers, O(1) Map-based routing
webhook/BotWebhookServer.java	HTTP server on :8081, POST /webhook/{roomId} for external bots
util/ChatFrameCodec.java	ENCODER: Message→JSON (Function), DECODER: JSON→Optional<ChatFrame> (Function)
ui/ChatClientUI.java	Swing GUI — lambda ActionListeners, JList rooms sidebar, search bar, admin panel

Table 3.3 — Project File Structure — 19 Java Files

3.5 Design Selection

Different design approaches were considered before selecting the final architecture.

- A peer-to-peer model was not suitable because it becomes difficult to manage as the number of users increases. It also creates challenges in handling group communication.
- An HTTP-based system was also not preferred because it requires continuous requests, which increases latency and reduces efficiency.

Layer	Technology	What It Does in FuncChat Pro
WS Server	Netty 4.1	Non-blocking NIO. Routes frames through dispatch table. Handles all client connections via a pipeline of ChannelHandlers.
Database	Firebase RT DB	Stores messages, rooms, users, and typing state as a JSON tree. Free Spark tier — zero billing. OkHttp4 async REST calls.
Authentication	Firebase Auth	Handles user registration and login. Returns a UID used as the session key throughout all service lookups.
File Storage	AWS S3	Stores file attachments privately. Delivers them via pre-signed URLs that expire after 15 minutes.
HTTP Client	OkHttp 4	Makes all Firebase REST API calls (GET, PUT, PATCH, DELETE) asynchronously on a shared thread pool.
JSON	Jackson 2	Serialises and deserialises all ChatFrame objects. Supports immutable models via @JsonCreator constructor injection.
Desktop UI	Java Swing	Full desktop client — rooms sidebar, typing label, file attach button, search bar, admin panel, message status
Testing	JUnit 5	40+ unit tests covering all FP constructs — Predicate, Function, Optional, Stream, BiPredicate. Fully offline.
Build	Maven + Docker	Fat JAR via maven-shade-plugin. Docker image with Eclipse Temurin 17 base, deployed to ECR.
CI/CD	GitHub Actions	On every push to main: run tests → build JAR → build Docker image → push to ECR → SSH into EC2 → restart container.
Cloud Hosting	AWS EC2 + ALB	t3.micro instance (free tier). Application Load Balancer handles SSL termination and WebSocket upgrade on port

Table 3.5 — Full Technology Stack

The WebSocket-based client-server model was chosen because it provides a persistent connection, allowing instant communication. It is efficient, scalable, and well-suited for real-time applications.

3.6 Implementation Plan/Methodology

The implementation of the system follows a three-tier architecture. The client side is developed using Java Swing, which provides the interface for users to interact with the system. The server side is built using Netty, which handles communication and processes requests efficiently. The data is managed using Firebase, while files are stored in AWS S3. The development was carried out in stages. First, the basic messaging system was implemented.

```

26
27 # AWS Credentials
28 S3_ACCESS_KEY=AKIATU2U0UAQ7TGFWR46
29 S3_SECRET_KEY=CnX37qVOG2hiCaBvYermmbbTH79tq0MzvPZnuWAM2
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Figure 3.4 DB Integration

Figure 3.5 AWS Cloud Deployment Architecture with CI/CD Pipeline

AWS Service	Role in FuncChat Pro
EC2 t3.micro	Runs the Docker container with the Netty server — free tier (750 hrs/month). Single instance is sufficient for team-scale use.
ALB	Application Load Balancer handles SSL termination on port 443 and performs the WebSocket (WSS) protocol upgrade for all clients.
ACM	AWS Certificate Manager provides a free SSL certificate for the custom domain, attached to the ALB.
Route 53	DNS service maps chat.yourdomain.com to the Load Balancer IP address.
S3 Bucket	Stores all file attachments privately. Files are delivered to clients via pre-signed URLs valid for 15 minutes.
Secrets Manager	Stores the Firebase API key and S3 credentials — injected as environment variables at container startup.
CloudWatch + SNS	Monitors EC2 CPU and memory. Sends email alert via SNS if CPU exceeds 70% for two consecutive evaluation periods.
GitHub Actions	On every git push to main: runs all 40+ tests → builds fat JAR → builds Docker image → pushes to ECR → SSH into EC2 → pulls new image → restarts

Table 3.6 — AWS Services and Their Roles

CHAPTER 4.

RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution

The implementation of the FuncChat Pro project was carried out using the Java programming language, selected for its robustness, object-oriented capabilities, and support for functional programming features introduced in Java 8. The primary objective of the implementation phase was to develop a real-time chat application that integrates modern communication technologies with functional programming concepts in a practical environment.

The system was designed using a modular approach, ensuring clear separation between the user interface, server-side logic, and data management components. This modular structure made the system easier to develop, test, and maintain, while also allowing independent handling of different functionalities such as messaging, file sharing, and database operations.

4.1.1 System Architecture Representation

The FuncChat Pro system follows a three-tier architecture consisting of the client layer, server layer, and data layer. The client layer is implemented using Java Swing, which provides an interactive graphical interface for users to send messages, join chat rooms, and share files. The server layer is built using Netty, a high-performance networking framework that handles WebSocket communication.

The data layer consists of Firebase Realtime Database and AWS S3. Firebase is used to store user data, chat messages, and room information, while AWS S3 is used for storing files securely. This separation ensures scalability and efficient data handling.

4.1.2 Message Processing and Communication Flow

The communication in FuncChat Pro is based on the WebSocket protocol, which allows continuous, bidirectional communication between the client and the server. When a user sends a message, it is first transmitted to the server through an open WebSocket connection.

The server then validates the message using predefined conditions, ensuring that it is not empty or invalid. After validation, the message is processed and stored in the Firebase database. Once stored, the server broadcasts the message to all users connected to the same chat room. This process happens almost instantly, creating a seamless real-time communication experience.

The message handling process is implemented using functional programming techniques, where different operations such as validation, transformation, and broadcasting are handled using lambda expressions and functional interfaces. This results in cleaner and more modular code.

4.1.3 File Upload and Storage Mechanism

The file sharing functionality in FuncChat Pro is implemented using AWS S3 for storage. When a user uploads a file, the system first performs validation checks, such as verifying the file type and size.

After successful validation, the file is uploaded to AWS S3 using secure pre-signed URLs. These URLs allow controlled access to the uploaded files, ensuring security and preventing unauthorized usage. Once the file is uploaded, a reference link is generated and shared with other users in the chat. This approach ensures efficient file handling without overloading the server.

4.1.4 Functional Programming Integration

A significant aspect of the implementation is the use of functional programming concepts in Java. These concepts were applied in multiple parts of the system to improve code quality and performance.

Lambda expressions were used to simplify event handling and message processing logic. Stream API was used for filtering and transforming data efficiently. Predicate functions were applied for validation tasks, such as checking message content or file conditions.

Asynchronous operations were handled using `CompletableFuture`, allowing the system to perform tasks without blocking the main execution flow. This was particularly useful in handling multiple user requests simultaneously.

The integration of these concepts resulted in a more concise, readable, and maintainable codebase.

4.1.5 Database Structure and Data Handling

The Firebase Realtime Database was structured in a hierarchical JSON format, allowing efficient storage and retrieval of data. The database consists of multiple nodes such as users, rooms, messages, and typing status.

Each node is designed to store specific information. For example, the users node stores user details, while the messages node stores chat history. This structured approach ensures quick access and real-time synchronization across all clients.

Whenever a new message is added or updated, Firebase automatically synchronizes the data across all connected users, eliminating the need for manual updates.

4.1.6 Deployment and CI/CD Pipeline

The deployment of FuncChat Pro was carried out using a CI/CD pipeline to automate the build and deployment process. GitHub Actions was used to trigger workflows whenever code changes were pushed to the repository.

The application was containerized using Docker, ensuring consistency across different environments. The Docker image was then deployed on AWS EC2, where the server runs continuously.

This automated pipeline reduced manual effort and ensured that updates could be deployed quickly and reliably.

4.1.7 Performance Metrics

To evaluate the performance of the system, several parameters were considered during testing.

The time taken for message delivery was observed to be very low, indicating efficient real-time communication. The system was able to handle multiple users simultaneously without noticeable delay.

The responsiveness of the user interface was also tested, and it remained smooth even during continuous message exchange. File uploads and downloads were completed successfully within acceptable time limits.

These observations confirm that the system performs efficiently under normal operating conditions.

4.1.8 Results and Observations

The results obtained from testing indicate that the system meets all functional requirements. Real-time messaging works effectively, with minimal latency. Group chat functionality allows multiple users to communicate simultaneously without issues.

File sharing operates reliably, with proper validation and secure storage. The use of Firebase ensures that all users receive updates instantly, maintaining data consistency.

It was also observed that the use of functional programming concepts improved the overall structure of the code, making it easier to understand and maintain.

4.1.9 Validation of Results

To validate the system, different test scenarios were executed. These included testing communication between multiple users, checking file upload functionality, and verifying database synchronization.

Special cases were also tested, such as sending empty messages, uploading unsupported file types, and handling network interruptions. In all cases, the system behaved as expected and handled errors appropriately.

The results were consistent across multiple test runs, confirming the reliability of the system.

4.1.10 Summary of Findings

The implementation of FuncChat Pro demonstrates that it is possible to build a scalable and efficient real-time communication system using modern technologies.

The system successfully integrates WebSocket communication, cloud-based storage, and functional programming concepts. Among all components, the use of WebSocket and asynchronous processing played a major role in achieving real-time performance.

CHAPTER 5.

CONCLUSION AND FUTURE WORK

5.1 Conclusion

The development of FuncChat Pro successfully demonstrates how a real-time communication system can be built using modern technologies while maintaining simplicity and efficiency. The project was designed with the aim of creating a practical application that not only solves a real-world problem but also showcases important programming concepts, particularly functional programming in Java.

Throughout the project, the system was developed in a structured manner, starting from requirement analysis to design, implementation, and testing. The use of a three-tier architecture helped in clearly separating different components of the system, making it easier to manage and extend. The client interface, server logic, and data storage were handled independently, which improved the overall organization of the application.

One of the key achievements of this project is the successful implementation of real-time communication using WebSocket technology. Messages are transmitted instantly between users, creating a smooth and responsive user experience. The system was also able to handle multiple users at the same time without significant performance issues, which shows that the chosen architecture is suitable for scalable applications.

Another important aspect of the project is the integration of cloud services. Firebase Realtime Database enabled real-time synchronization of data, ensuring that all users receive updates instantly. AWS S3 was used for file storage, allowing users to share files securely and efficiently. These integrations demonstrate how cloud technologies can be used to enhance application functionality.

The project also effectively incorporates functional programming concepts such as lambda expressions, streams, predicates, and asynchronous processing. These concepts helped in writing cleaner and more modular code, making the system easier to understand and

maintain. The use of asynchronous processing ensured that the application remains responsive even when handling multiple operations.

Testing and validation confirmed that the system performs reliably under normal conditions. All major features, including messaging, group chat, and file sharing, were implemented successfully and worked as expected. The deployment process using a CI/CD pipeline further ensured that the application can be maintained and updated efficiently.

Overall, FuncChat Pro achieves its intended objectives and serves as a good example of how modern software development techniques can be applied to build a real-time communication system. It not only provides practical functionality but also strengthens understanding of key programming concepts.

5.2 Future Work

Although the current version of FuncChat Pro is fully functional and meets the project requirements, there are several areas where the system can be improved and extended in the future. One possible enhancement is the addition of end-to-end encryption to improve the security of messages and file transfers. This would ensure that only the intended users can access the data, making the system more suitable for sensitive communication.

Another improvement could be the development of a web-based or mobile version of the application. Currently, the system is limited to a desktop interface, but expanding it to other platforms would increase its usability and reach.

The system can also be enhanced by adding multimedia communication features such as voice and video calls. This would transform the application into a more comprehensive communication platform.

From a performance perspective, the system can be scaled further using distributed architectures and load balancing techniques. This would allow the application to handle a larger number of users efficiently.

Additional features such as notifications, user status indicators, and chat history backup can also be included to improve the user experience.

Furthermore, the functional programming aspects of the system can be extended by incorporating more advanced techniques and optimizing existing implementations.

In conclusion, while FuncChat Pro provides a strong foundation for a real-time communication system, there is significant scope for future development. These improvements can make the system more secure, scalable, and feature-rich, bringing it closer to industry-level applications.

REFERENCES

1. Warburton, R. (2014). Java 8 Lambdas: Pragmatic Functional Programming. O'Reilly Media.
2. Subramaniam, V. (2014). Functional Programming in Java: Harnessing the Power of Java 8 Lambda Expressions. Pragmatic Bookshelf.
3. Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley. [Items on immutability, Optional, and functional interfaces]
4. Tyson, M. (2023). Java Concurrency and Multithreading in Practice. InfoQ.
5. Netty Project Contributors. (2023). Netty 4.1 Reference Documentation. <https://netty.io/wiki/user-guide-for-4.x.html>
6. Google Firebase Team. (2024). Firebase Realtime Database Documentation. <https://firebase.google.com/docs/database>
7. Google Firebase Team. (2024). Firebase Authentication Documentation. <https://firebase.google.com/docs/auth>
8. Amazon Web Services. (2024). Amazon S3 Developer Guide — Pre-signed URLs. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/ShareObjectPreSignedURL.html>
9. Amazon Web Services. (2024). AWS EC2 User Guide — Application Load Balancer for WebSockets. <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/>
10. JUnit Team. (2024). JUnit 5 User Guide. <https://junit.org/junit5/docs/current/user-guide/>
11. Docker, Inc. (2024). Dockerfile Reference. <https://docs.docker.com/engine/reference/builder/>
12. GitHub, Inc. (2024). GitHub Actions Documentation — Workflows and Jobs. <https://docs.github.com/en/actions>
13. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
14. Oracle Corporation. (2024). Java SE 17 API Specification — java.util.function package. <https://docs.oracle.com/en/java/api/java.base/java/util/function/>
15. Square, Inc. (2023). OkHttp 4 Documentation. <https://square.github.io/okhttp/>
16. FasterXML. (2024). Jackson Databind Documentation — @JsonCreator and Immutable Models. <https://github.com/FasterXML/jackson-databind>

APPENDIX — USER MANUAL

Complete Step-by-Step Instructions to Run FuncChat Pro

A.1 Prerequisites

Before running FuncChat Pro, ensure the following software is installed on your machine: Java Development Kit (JDK) 17 or later, Apache Maven 3.8 or later, Git (for cloning the repository), and a Firebase account (free). An AWS account is only required for production deployment — local development runs entirely without AWS.

A.2 Firebase Setup

Step 1: Open a web browser and navigate to console.firebase.google.com. Sign in with your Google account. Step 2: Click 'Add project', enter a project name such as `funcchat-pro`, and follow the prompts to create the project. Step 3: In the left sidebar, click 'Build' then 'Realtime Database'. Click 'Create Database', select a server location, and choose 'Start in test mode' for local development. Step 4: Click the gear icon next to 'Project Overview' and select 'Project settings'. Scroll to the 'Your apps' section and copy the `apiKey` and `projectId` values. Step 5: In the left sidebar, click 'Build' then 'Authentication'. Click 'Get started'. Enable the 'Email/Password' sign-in provider.

A.3 Configuration

Step 6: Clone the repository to your local machine using the command: `git clone [repository-url]`. Navigate into the project directory. Step 7: Copy the environment template file: `cp .env.template .env`. Open the `.env` file in any text editor and paste the Firebase `apiKey` and `projectId` values you copied in Step 4. Also add your Firebase `databaseURL` — this is in the format `https://your-project-id-default-rtdb.firebaseio.com`.

A.4 Building and Running Locally

Step 8: Build the server JAR by running: `mvn clean package -DskipTests`. This creates a file named `funcchat-pro-server.jar` in the `target` directory. Step 9: Start the server by running: `java -jar target/funcchat-pro-server.jar`. You should see log output indicating that the Netty server has started on port 8080.

Step 10: Open a new terminal window. Run the desktop client by executing: `mvn exec:java -Dexec.mainClass=com.chatapp.ui.ChatClientUI`. The Swing GUI window will open. Step 11: Open a second terminal window and run the same command to start a second client instance. You now have two independent chat clients.

A.5 Using the Application

Step 12: In each client window, enter an email address, a password, and a display name, then click 'Sign In'. If registering for the first time, check 'New user' before clicking. Step 13: In one client, click the '+' button in the Rooms panel on the left side of the window. Enter a room name and click 'Create'. Step 14: In the second client, click 'Refresh' in the Rooms panel to see the newly created room. Click on the room name to join it. Step 15: Both users are now in the same room. Type a message in the input field at the bottom of the window and press Enter or click 'Send'. The message will appear in both client windows in real time.

Step 16: To share a file, click the paper clip icon next to the message input field. Select a file up to 5 MB. The file will be uploaded to AWS S3 (if configured) and a clickable download link will appear in the chat. Step 17: To search messages, type in the search bar at the top of the message panel. Matching messages will be highlighted. Step 18: To run all unit tests, execute: `mvn test` in the project directory. All 47 tests should pass within a few seconds.

A.6 Production Deployment (AWS)

For production deployment, a Dockerfile and a GitHub Actions workflow file (`.github/workflows/deploy.yml`) are included in the repository. Set the following secrets in your GitHub repository settings: `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `ECR_REPOSITORY`, `EC2_HOST`, `EC2_USERNAME`, `EC2_SSH_KEY`, and all Firebase and S3 environment variables. After configuring these secrets, every push to the main branch will automatically run tests, build a Docker image, push it to AWS ECR, and deploy it to your EC2 instance via SSH.